
boolean



Express Js

Cos'è?

Express Js - Che cos'è?

Come ogni linguaggio che si rispetti, **Node.js** ha un grande potenziale e permette di gestire ed avviare un server. Questo però non sempre è facile, sopratutto se l'applicazione è un po' più complessa.

Per questo motivo qualcuno ha pensato bene di creare un framework che semplificasse le operazioni più comuni.

E così è nato **Express Js** o semplicemente **Express**.

Express Js - Che cos'è?

Come ogni linguaggio che si rispetti, **Node.js** ha un grande potenziale e permette di gestire ed avviare un server. Questo però non sempre è facile, sopratutto se l'applicazione è un po' più complessa.

Per questo motivo qualcuno ha pensato bene di creare un framework che semplificasse le operazioni più comuni.

E così è nato **Express Js** o semplicemente **Express**.



Framework backend minimalista e veloce che offre funzionalità e strumenti solidi per lo sviluppo di applicazioni di backend scalabili.

Offre un sistema di routing e funzioni semplificate per estendere il framework sviluppando componenti e parti più potenti in base ai casi d’uso dell’applicazione.

Express Js - Chi?

Express.js è stato fondato da **TJ Holowaychuk**. La prima versione, secondo il repository di Express.js su GitHub, risale al **22 maggio 2010**, versione 0.12.

Nel giugno 2014, i diritti per gestire il progetto sono stati acquisiti da **StrongLoop**. Questa è stato poi acquisito da **IBM** nel settembre 2015.

Nel gennaio 2016, IBM ha annunciato che avrebbe posto Express.js sotto la supervisione dell'incubatore **Node.js Foundation**.





Express Js

Setup

Express Js - Installazione

1. Creiamo la cartella del nostro progetto
2. Dentro la cartella appena creata inizializziamo la cartella con npm

```
1 $ pnpm init
```

3. Quando ci verrà chiesto di specificare l'**entry-point** inseriamo **`app.js`**

```
version: (1.0.0)
description:
entry point: (index.js) app.js
```

4. Una volta inizializzato il progetto, installiamo express

```
1 $ pnpm add express
```

https://expressjs.com/en/starter/installing.html

Express Js - Hello World

Una volta fatte le configurazioni base, possiamo ora creare il nostro primo server Express.

1. Creiamo un file **app.js** e scriviamo

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
```

https://expressjs.com/en/starter/hello-world.html

Express Js - Hello World

2. Nel file **package.json** aggiungiamo uno script per avviare il file app.js

```
"scripts": {
  "start": "node app.js",
  "watch": "node --watch app.js"
},
```

3. Avviamo il nostro server

```
$ pnpm watch
```

https://expressjs.com/en/starter/hello-world.html

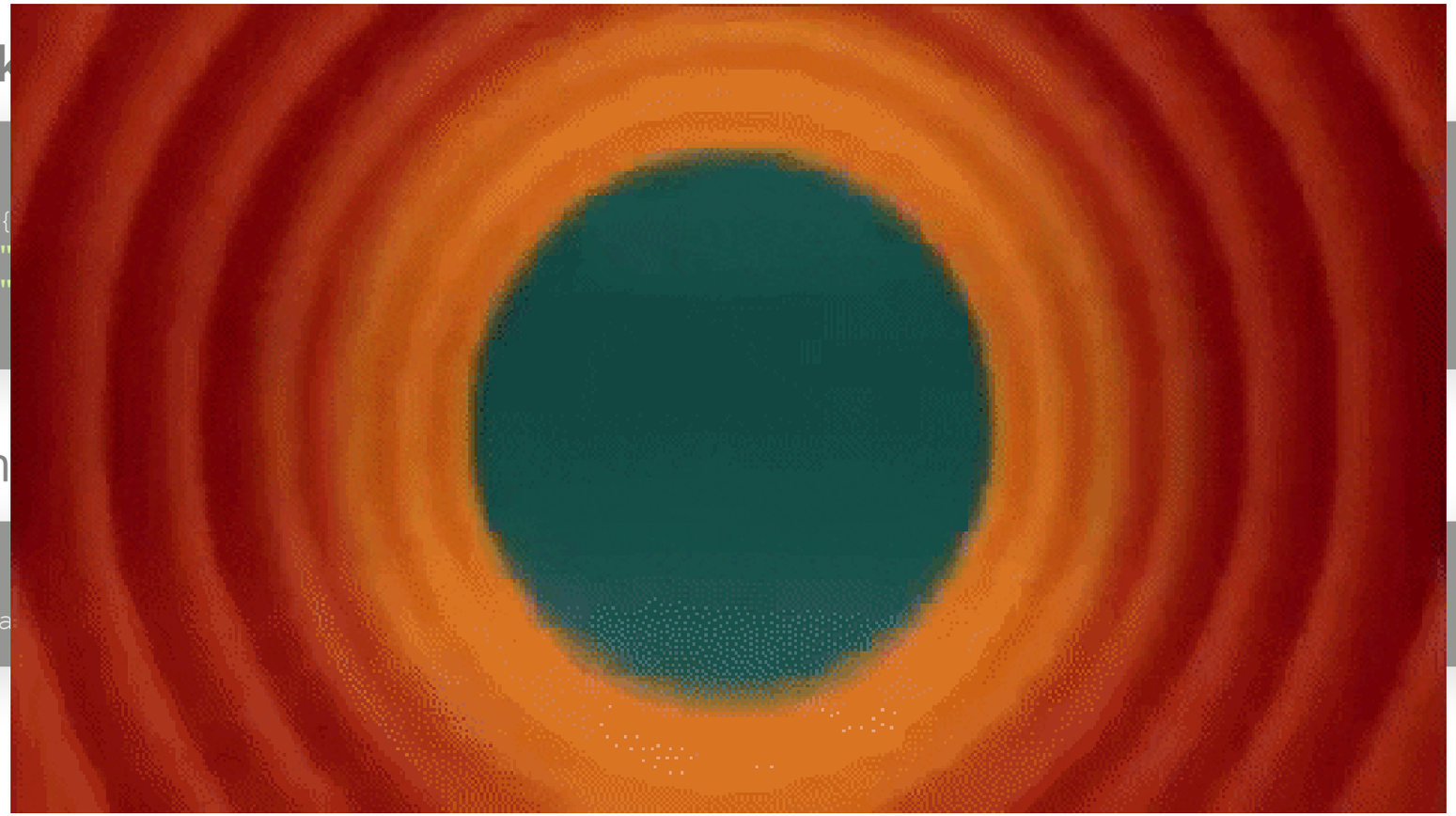
Express Js - Hello World

2. Nel file **pack**

```
"scripts": {
  "start": "
  "watch": "
},
```

3. Avviamo il n

```
$ npm run wa
```



https://expressjs.com/en/starter/hello-world.html



Postman App

Testiamo il nostro server

https://www.getpostman.com/downloads/



Express Js

Come funziona?

Express Js - Come funziona

Ma cosa fa il codice che abbiamo inserito in **app.js**?

Analizziamolo:

1. importiamo express
2. inizializziamo express invocandolo come una funzione e salvando il risultato in una variabile
3. definiamo la *prima rotta*
4. avviamo il server, mettendolo in ascolto sulla porta indicata



Express Js - app

Che cos'è la variabile **app**?

Questa variabile conterrà un oggetto di Express e avrà quindi al suo interno tutta una serie di metodi che possiamo invocare per eseguire varie operazioni tipiche dei server come:

- Rotte HTTP
- Configurazione middleware
- Rendering view in HTML
- Registrazione di un template engine
- ... molto altro

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
```

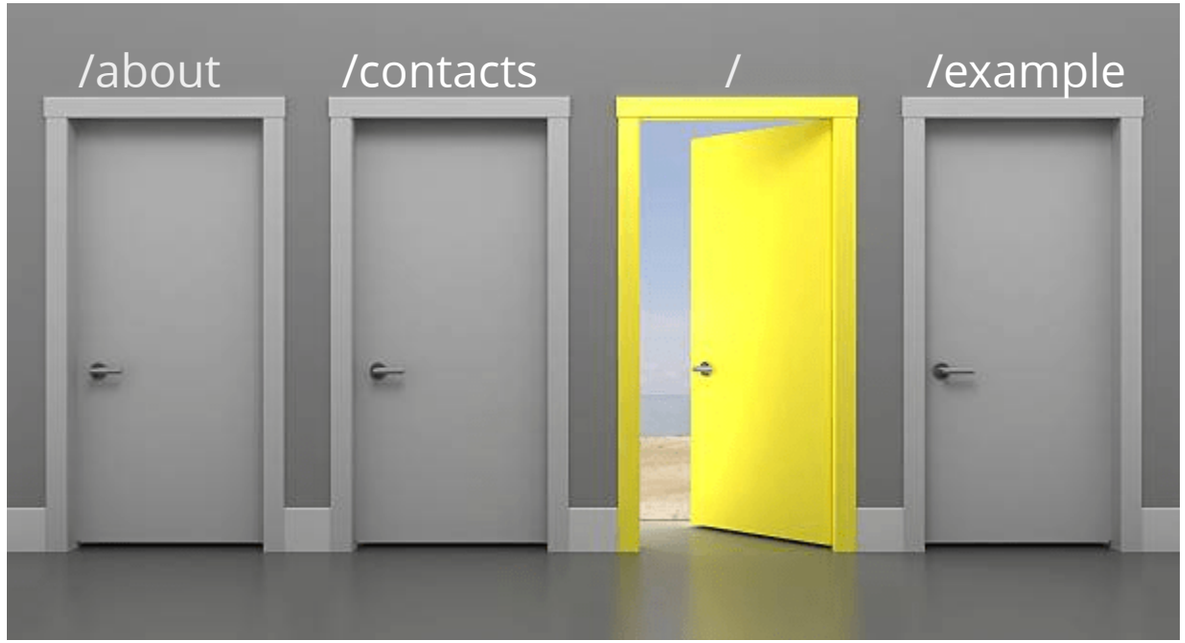
Rotte

Rotte

In Express JS, le *rotte* sono come indirizzi per indicare al server cosa fare quando riceve una richiesta su un certo percorso (URL).

Immagina un sito come una casa con diverse stanze: le rotte sono come le porte che portano a quelle stanze.

Ogni rotta è associata a un'azione specifica che il server deve compiere, ad esempio mostrare una pagina o inviare dei dati.



Solitamente, c'è una rotta il cui URL è solo uno "/". Quella è la home della nostra applicazione Express

Express Js - Rotte

Tramite Express possiamo definire tutti gli indirizzi (detti **rotte)** di cui abbiamo bisogno per la nostra applicazione. Tramite l'oggetto **app**, per ogni rotta possiamo specificare 3 elementi:

1. **metodo HTTP** che dovrà essere usato per accedere alla rotta (per il momento *get*)
2. **path** ossia l'URL della rotta
3. **funzione** che ha lo scopo di rispondere quando viene richiamata la rotta

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
```



Request & Response

Ovvero: come funziona il web

HTTP

Nel mondo informatico la maggior parte delle comunicazioni avviene tramite il protocollo **HTTP**, ovvero *Hypertext Transfer Protocol.* Questo viene utilizzato dal browser per la trasmissione delle pagine web (HTML) e di una serie di altri formati correlati (script, immagini, etc).

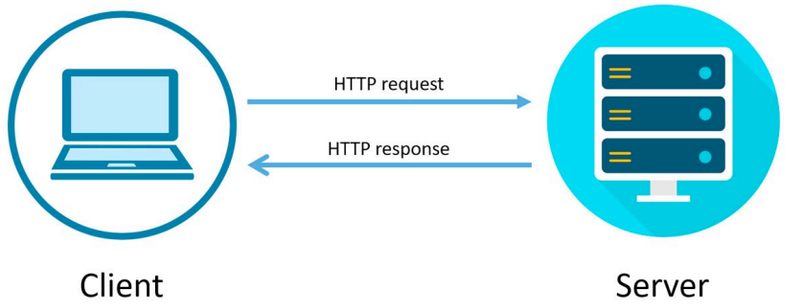
Quando è presente la **S** finale (Secure) la comunicazione avviene con una **connessione criptata** ottenuta sfruttando crittografia asimmetrica tramite Transport Layer Security (TLS) e relativo certificato installato sul server.



Request e Response

Il protocollo HTTP prevede 2 fasi:

Innanzitutto, un client (ad esempio un browser) fa una **Richiesta** al server ad una rotta specifica, per farsi restituire dei dati o delle intere pagine HTML.



Successivamente, il server riceve la richiesta e individua a quale rotta è stata fatta. Ogni rotta ha una funzione dedicata e questa funzione deve restituire una **Risposta** al client, fornendo un testo, dei dati o intere pagine!

HTTP

HTTP Request

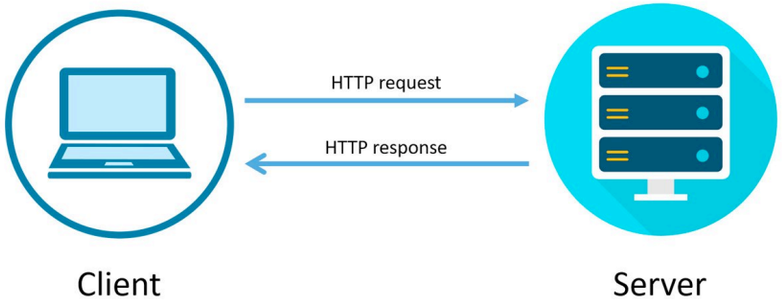
Chiamata che il client/browser effettua al server web. E' composta dai seguenti elementi:

- Method (GET, POST, PUT, PATCH, DELETE, ecc)
- URL (comprensiva di eventuale **query string**)
- Request Headers
- Request Body (dati inviati al server)

HTTP Response

Risposta che il server invia al client/browser a seguito di ciascuna request. E’ composta dai seguenti elementi:

- Status Code (es. 200 - OK, oppure 404 - Page Not Found, etc)
- Response Headers
- Response Body (dati inviati al client/browser)



Express Js - Request e Response

In Express, la request e la response, vengono fornite automaticamente come parametri alla funzione che gestisce una rotta:

req: (request) oggetto che rappresenta la richiesta e contiene delle proprietà per accedere alle query string, parametri dinamici, dati ricevuti e molto altro.

res: (response) oggetto che rappresenta la risposta che l'applicazione Express invia a seguito di una richiesta. Permette di specificare i dati da inviare, la loro tipologia o altre operazioni.

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
```

Per ora concentriamoci sulle Response



Express Js - Response

Come ogni server che si rispetti dobbiamo essere in grado di rispondere in diversi modi e di inviare dati o pagine html.

Tramite l'oggetto **res** che ogni funzione di ogni rotta riceve, possiamo invocare vari metodi:

- res.**send()** - invia un contenuto testuale/ html
- res.**json()** - invia i dati sotto forma di json
- res.**redirect()** - esegue un redirect verso un altra pagina
- res.**download()** - invia un file e ne effettua il download tramite il browser
- res.**sendFile()** - invia un file che verrà aperto direttamente nel browser

Express Js - Response

Come si usano questi metodi?

Ogni metodo richiederà degli argomenti specifici, quindi è essenziale consultare la documentazione di ogni metodo per capire come usarlo.

```
1   // risponde con Content-Type text/html
2   res.send(`<h1>Hello world</h1>`);
3
4   // risponde con Content-Type: application/json
5   res.json({ message: "Hello world" });
6
7   // risponde con un redirect alla pagina /altra_pagina
8   res.redirect("/altra_pagina")
```

Express Js - Response

Abbiamo detto che Express utilizza un metodo dell'oggetto **res** che si chiama **res.send()** e che questo restituisce del testo (che può anche contenere tag HTML)

Se necessario, possiamo però specificare il tipo di dato che stiamo restituendo tramite il metodo **res.type()**

Nell'esempio a fianco, stiamo esplicitando il **type("html")** anche se non sarebbe necessario, visto che è il tipo di default.

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.type("html")
7      .send(`<h1>Titolo di una pagina HTML!<h1>`)
8  })
9
10 app.listen(port, () => {
11   console.log(`Example app listening on port ${port}`)
12 })
```

Express Js - Response

Quando costruiamo un'API, probabilmente abbiamo bisogno di restituire un JSON.

Naturalmente possiamo farlo cambiando l'argomento fornito nel metodo **type!**

Notiamo che l'oggetto person nell'esempio viene convertito in JSON automaticamente.

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    const person = {
7      name: "Ted",
8      lastname: "Lasso"
9    };
10
11   res.type('json').send(person);
12 })
13
```



Attenzione: quando impostiamo il type('json'), il metodo send() converte in JSON array e oggetti, ma non altri tipi di dato!

Express Js - Response

Come visto nella prima slide, Express ci mette anche a disposizione il metodo **res.json()** dell'oggetto response.

Questo metodo non è altro che una scorciatoia: imposta automaticamente il type su "json" e invia la risposta, convertendo anche in questo caso il tipo di risposta.

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    const person = {
7      name: "Ted",
8      lastname: "Lasso"
9    };
10
11   res.json(person);
12 })
13
```



Utilizzando il metodo res.json() tutti i tipi di dato vengono automaticamente convertiti da JS a JSON!

Express Js - Risposte

Methods

res.append()
res.attachment()
res.cookie()
res.clearCookie()
res.download()
res.end()
res.format()
res.get()
res.json()
res.jsonp()
res.links()
res.location()
res.redirect()
res.render()
res.send()
res.sendFile()
res.sendStatus()
res.set()
res.status()
res.type()
res.vary()

Ci sono molti altri metodi che possiamo usare, pertanto facciamo sempre riferimento alla **documentazione ufficiale**.

Questa è una best-practice da utilizzare per qualsiasi tecnologia che vogliamo imparare!

https://expressjs.com/en/5x/api.html#res

Express Js - Risposte

Methods

res.append()
res.attachment()
res.cookie()
res.clearCookie()
res.download()
res.end()
res.format()
res.get()
res.json()
res.jsonp()
res.links()
res.location()
res.redirect()
res.render()
res.send()
res.sendFile()
res.sendStatus()
res.set()
res.status()
res.type()
res.vary()

Ci sono molti altri metodi che possiamo usare, pertanto facciamo sempre riferimento alla **documentazione ufficiale**.

Questa è una best-practice da utilizzare per qualsiasi tecnologia che vogliamo imparare!

https://expressjs.com/en/5x/api.html#res

Ah, cosa molto importante:

Consultiamo la documentazione ufficiale!!

L'abbiamo già detto? Boh!?





Asset Statici

Express Js - Asset Statici

Un server non si limita soltanto a restituire e manipolare testi e dati.

Spesso dobbiamo poter leggere interi file statici (immagini, file css, js o altro)

Express di default non attiva questo servizio, pertanto lo dobbiamo attivare noi.

Tramite la funzione **app.use()** possiamo richiedere ad Express di "abilitare" alcune funzionalità extra!

```
1  const express = require("express");
2  const app = express();
3  const port = 3000;
4
5  // Registriamo funzionalità extra
6  app.use(???);
7
8  app.get("/", (req, res) => {
9    // codice funzione
10 })
11
12 app.listen(port, () => {
13   console.log(`Example app listening on port ${port}`);
14 });
```

Express Js - Asset Statici

A questa funzione passiamo come argomento il middleware nativo di express invocando la funzione **express.static()**.

Alla funzione static indichiamo come argomento il **nome della cartella** che conterrà i nostri file statici.

```
1  const express = require("express");
2  const app = express();
3  const port = 3000;
4
5  app.use(express.static('public'));
6
7  app.get("/", (req, res) => {
8    // codice funzione
9  })
10
11 app.listen(port, () => {
12   console.log(`Example app listening on port ${port}`);
13 });
```

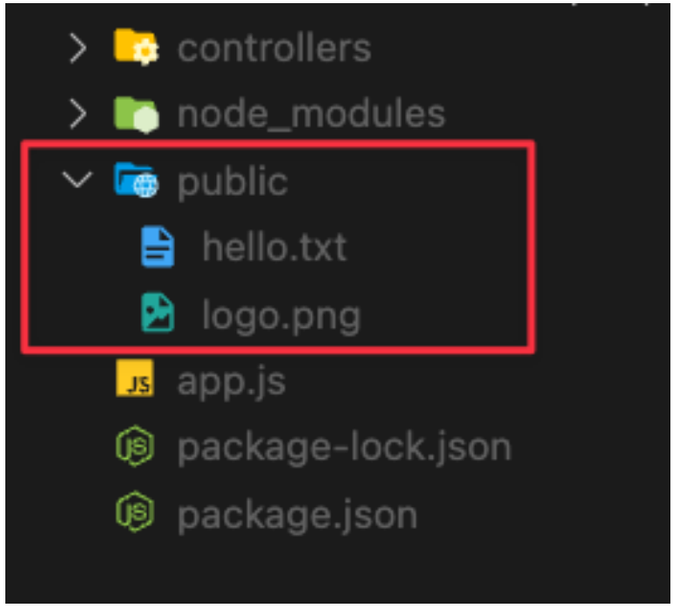
Express Js - Asset Statici

Infine nella cartella **public** inseriamo qualsiasi file vogliamo utilizzare.

Questi saranno accessibili scrivendo direttamente il dominio del nostro sito seguito dal nome del file.

```
http://localhost:3000/hello.txt
```

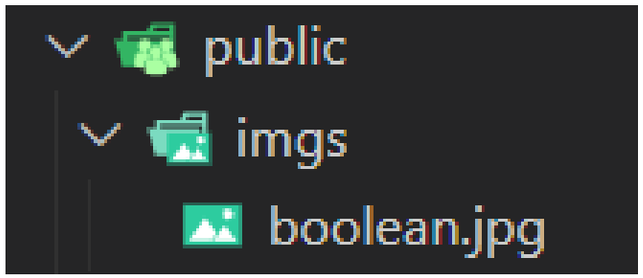
```
http://localhost:3000/logo.png
```



Express Js - Asset Statici

Adesso, se vogliamo visualizzare le immagini, basterà inserire il percorso, considerando la partenza dalla cartella **public**

Quindi se avessimo ad esempio:



Dovremmo scrivere sulla barra del browser:

```
localhost:3000/imgs/boolean.jpg
```

Se rispondessimo HTML dovremmo invece scrivere:

```
<img src="imgs/boolean.jpg">
```

LIVE CODING



ESERCITAZIONE